Experiment-10

Write A Python Program To Implement A* Algorithm

Aim

To write and execute a Python program to implement the A (A-Star) Search Algorithm* to find the lowest-cost path from a starting node to a goal node using actual cost and heuristic values.

Algorithm

1. Start the program.
2. Define the graph with nodes and edge costs.
3. Assign heuristic values to each node.
4. Select the starting node and goal node.
5. Initialize the open list with the starting node.
6. Calculate the evaluation function \(f(n)=g(n)+h(n)\).
7. Select the node with the lowest evaluation value.
8. Check whether the selected node is the goal node.
9. If it is the goal node, return the path and total cost.
10. Otherwise, explore its neighboring nodes.
11. Update the path costs and evaluation values, and repeat until the goal is reached or no path remains.
12. Display the path and total cost, then stop.

Code

ex-10.py - D:/ex-10.py (3.10.0)

File Edit Format Run Options Window Help

```
# Program to implement A* Search Algorithm

def astar(graph, heuristic, start, goal):
    open_list = [(heuristic[start], 0, start, [start])]
    visited = set()

    while open_list:
        open_list.sort()
        f, g, node, path = open_list.pop(0)

        if node == goal:
            return path, g

        if node in visited:
            continue

        visited.add(node)

        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]

                open_list.append(
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    return None, float('inf')
```

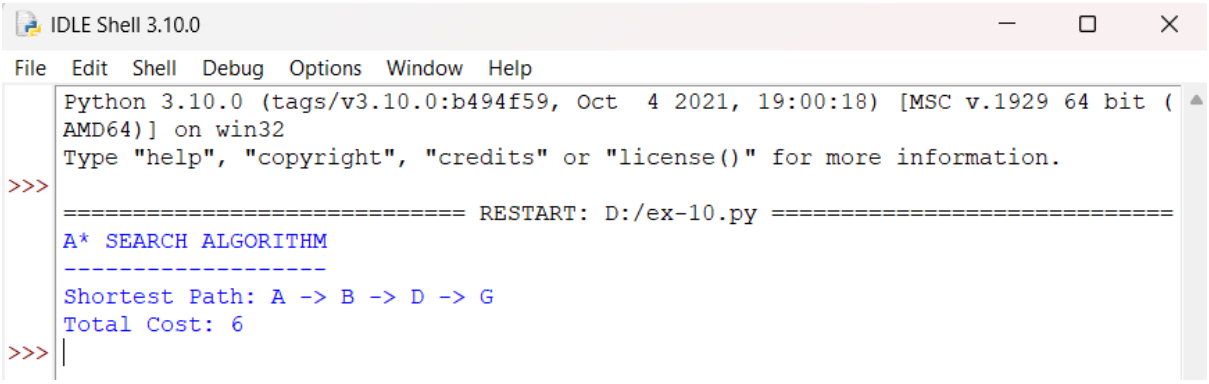
```
# Graph with edge costs
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [('G', 2)],
    'E': [('G', 4)],
    'F': [('G', 2)],
    'G': []
}

# Heuristic values
heuristic = {
    'A': 6,
    'B': 5,
    'C': 4,
    'D': 2,
    'E': 3,
    'F': 2,
    'G': 0
}

# Find the path
path, cost = astar(graph, heuristic, 'A', 'G')

if path:
    print("A* SEARCH ALGORITHM")
    print("--------------------")
    print("Shortest Path:", " -> ".join(path))
    print("Total Cost:", cost)
else:
    print("No path exists.")
```

Output



Result

The A Search Algorithm was successfully implemented . The program finds the path A → B → D → G with a total cost of 6 units.